Modulo 7: Creación y manipulación de tablas

Sistema de gestión de ventas

Estudiante: Marion Cordova Canelo

Introducción

La empresa comercial ficticia **“Comercio Ya”** necesita implementar un sistema básico de gestión de ventas que permita registrar clientes, productos y transacciones en una base de datos relacional. Actualmente la empresa utiliza planillas, lo que genera errores y dificulta el análisis de la información.

Para solucionar este problema, se propone desarrollar una base de datos relacional utilizando **SQL**, que permita almacenar, consultar y manipular información de forma eficiente.

Las bases de datos relacionales

Una base de datos relacional es un sistema de almacenamiento que organiza los datos en tablas relacionadas entre sí mediante claves y claves foráneas.

Cada tabla contiene:

- Filas (registro): Representan una instancia de los datos
- Columnas (atributos): Describen las características de cada registro

Este modelo fue propuesto por Edgar F. Codd en 1970 y es uno de los más utilizados en sistemas empresariales.

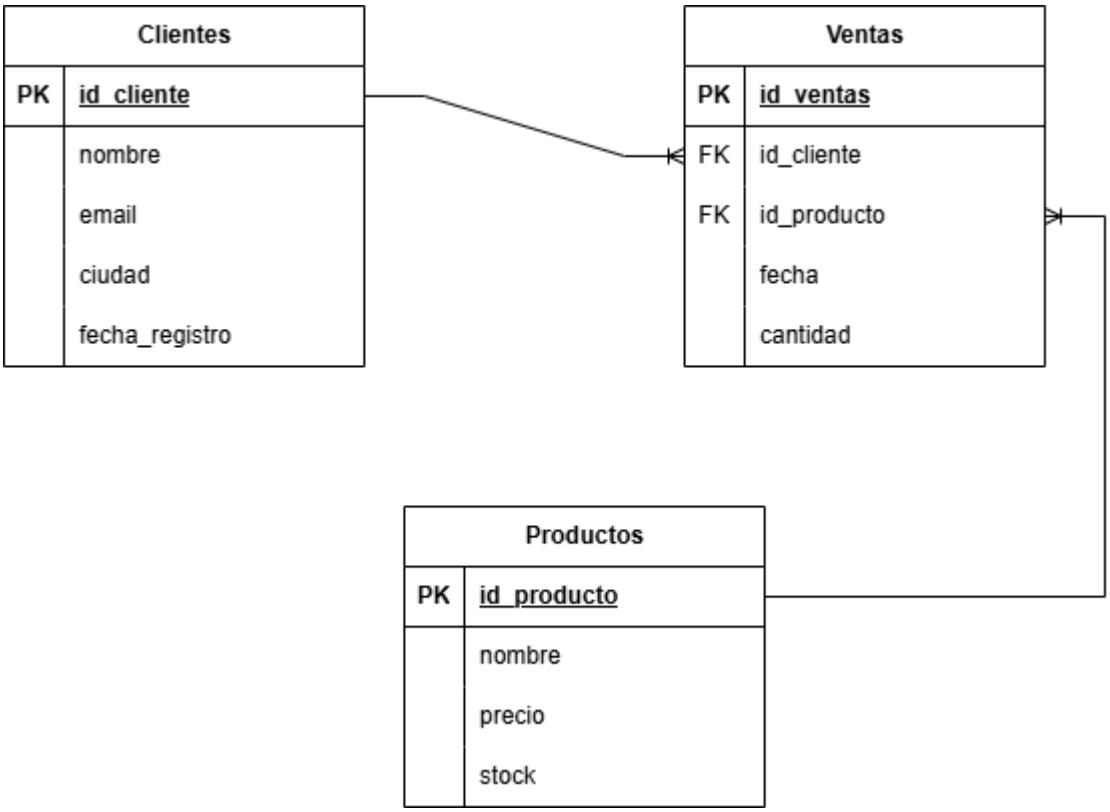
Ventajas frente al uso de plantilla

CARACTERISTICAS	PLANTILLAS (EXCEL)	BASE DE DATOS RELACIONAL
Organización	Limitada	Estructurada
Integridad de los datos	Baja	Alta
Relaciones entre los datos	Difícil	Nativa
Escalabilidad	Limitada	Alta
Consultas complejas	Difícil	Fácil mediante SQL

Las Base de datos permiten:

- Evitar la duplicación de los datos
- Mejorar la integridad de la información
- Realizar análisis complejos de manera eficiente

Diagrama Entidad Relación



El diagrama entidad–relación del sistema de gestión de ventas representa tres entidades principales: Clientes, Productos y Ventas. La entidad Ventas actúa como tabla transaccional y vincula a los clientes con los productos mediante claves foráneas. De este modo, el modelo permite registrar qué cliente compró qué producto, en qué fecha y en qué cantidad, manteniendo la integridad referencial y una estructura relacional simple, clara y funcional.

Creación de Base de datos

Creación de la tabla Clientes

```sql
-- Crear tabla clientes
CREATE TABLE Clientes (
    id_cliente INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    ciudad TEXT,
    fecha_registro TEXT
);
```

Con 10 registros

```sql
-- Insertar datos
INSERT INTO Clientes (nombre, email, ciudad, fecha_registro) VALUES
 ('Ana Torres','ana.torres@gmail.com','Santiago','2024-01-10'),
 ('Juan Perez','juan.perez@gmail.com','Valparaiso','2024-02-05'),
 ('Maria Lopez','maria.lopez@gmail.com','Concepcion','2024-03-12'),
 ('Carlos Soto','carlos.soto@gmail.com','Santiago','2024-01-22'),
 ('Camila Rojas','camila.rojas@gmail.com','Temuco','2024-02-18'),
 ('Pedro Gonzalez','pedro.gonzalez@gmail.com','Santiago','2024-03-02'),
 ('Daniela Muñoz','daniela.munoz@gmail.com','Valdivia','2024-01-30'),
 ('Luis Herrera','luis.herrera@gmail.com','Antofagasta','2024-02-14'),
 ('Valentina Diaz','valentina.diaz@gmail.com','Puerto Montt','2024-03-20'),
 ('Sebastian Morales','sebastian.morales@gmail.com','Rancagua','2024-02-27');
```

Tabla de Clientes

id_cliente	nombre	email	ciudad	fecha_registro
1	Ana Torres	ana.torres@gmail.com	Santiago	2024-01-10
2	Juan Perez	juan.perez@gmail.com	Valparaiso	2024-02-05
3	Maria Lopez	maria.lopez@gmail.com	Concepcion	2024-03-12
4	Carlos Soto	carlos.soto@gmail.com	Santiago	2024-01-22
5	Camila Rojas	camila.rojas@gmail.com	Temuco	2024-02-18
6	Pedro Gonzalez	pedro.gonzalez@gmail.com	Santiago	2024-03-02
7	Daniela Muñoz	daniela.munoz@gmail.com	Valdivia	2024-01-30
8	Luis Herrera	luis.herrera@gmail.com	Antofagasta	2024-02-14
9	Valentina Diaz	valentina.diaz@gmail.com	Puerto Montt	2024-03-20
10	Sebastian Morales	sebastian.morales@gmail.com	Rancagua	2024-02-27

Consultas simples

```sql
-- Consultas
SELECT * FROM Clientes;

SELECT * FROM Clientes
WHERE ciudad = 'Santiago';

SELECT * FROM Clientes
WHERE nombre LIKE '%Juan%';
```

1. Selecciona todos los datos de la tabla clientes
2. Selecciona solamente los datos de la tabla clientes, donde la ciudad es igual a “Santiago”
3. Selecciona únicamente los datos de la tabla clientes, donde nombre es “Juan”.

Creación de la tabla Productos y Ventas

```sql
-- Crear tabla productos
CREATE TABLE Productos (
    id_producto INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL,
    precio NUMERIC
);

-- Crear tabla ventas
CREATE TABLE Ventas (
    id_venta INTEGER PRIMARY KEY,
    id_cliente INTEGER,
    id_producto INTEGER,
    fecha TEXT,
    cantidad INTEGER,

    FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente),
    FOREIGN KEY (id_producto) REFERENCES Productos(id_producto)
);
```

Insertar datos en las tablas

```sql
-- Insertar datos en las tablas
INSERT INTO Productos (nombre, precio) VALUES
('Laptop',800000),
('Mouse',15000),
('Teclado',25000),
('Monitor',180000),
('Audifonos',35000),
('Webcam',45000),
('Impresora',120000),
('Tablet',300000),
('Disco SSD',90000),
('Silla Gamer',150000);
```

```sql
INSERT INTO Ventas (id_cliente, id_producto, fecha, cantidad) VALUES
(1,1,'2025-04-01',1),
(2,5,'2025-04-02',2),
(3,2,'2025-04-03',1),
(4,8,'2025-04-04',1),
(5,6,'2025-04-05',3),
(6,7,'2025-04-06',1),
(7,9,'2025-04-07',1),
(8,3,'2025-04-08',2),
(9,2,'2025-04-09',1),
(10,10,'2025-04-10',1),
(1,9,'2025-04-11',1),
(2,4,'2025-04-12',2),
(3,7,'2025-04-13',1),
(4,6,'2025-04-14',1),
(5,1,'2025-04-15',2),
(6,2,'2025-04-16',1),
(7,3,'2025-04-17',1),
(8,5,'2025-04-18',2),
(9,7,'2025-04-19',1),
(10,1,'2025-04-20',1);
```

Consultas

Se ejecuta una consulta con JOIN para poder obtener, que cliente compro que producto y cuando.

```sql
SELECT
c.nombre AS cliente,
p.nombre AS producto,
v.fecha
FROM Ventas v
JOIN Clientes c ON v.id_cliente = c.id_cliente
JOIN Productos p ON v.id_producto = p.id_producto;
```

Lo cual obtenemos la siguiente tabla:

cliente	producto	fecha
Ana Torres	Laptop	2025-04-01
Juan Perez	Audifonos	2025-04-02
Maria Lopez	Mouse	2025-04-03
Carlos Soto	Tablet	2025-04-04
Camila Rojas	Webcam	2025-04-05
Pedro Gonzalez	Impresora	2025-04-06
Daniela Muñoz	Disco SSD	2025-04-07
Luis Herrera	Teclado	2025-04-08
Valentina Diaz	Mouse	2025-04-09
Sebastian Morales	Silla Gamer	2025-04-10
Ana Torres	Disco SSD	2025-04-11
Juan Perez	Monitor	2025-04-12
Maria Lopez	Impresora	2025-04-13
Carlos Soto	Webcam	2025-04-14
Camila Rojas	Laptop	2025-04-15
Pedro Gonzalez	Mouse	2025-04-16
Daniela Muñoz	Teclado	2025-04-17
Luis Herrera	Audifonos	2025-04-18
Valentina Diaz	Impresora	2025-04-19
Sebastian Morales	Laptop	2025-04-20

Consultas agrupadas

Aplicar funciones de agregación sobre los datos

- Total de ventas: **SELECT SUM(cantidad) FROM ventas;**
  - **SUM → 27**
- Promedio de ventas : **SELECT AVG(cantidad) FROM Ventas;**
  - **AVG → 1.35**
- Total de ventas por cliente**:**

  SELECT c.nombre, SUM(v.cantidad) AS total_compras
  FROM Ventas v
  JOIN Clientes c ON v.id_cliente = c.id_cliente
  GROUP BY c.nombre;

nombre	total_compras
Ana Torres	2
Camila Rojas	5
Carlos Soto	2
Daniela Muñoz	2
Juan Perez	4
Luis Herrera	4
Maria Lopez	2
Pedro Gonzalez	2
Sebastian Morales	2
Valentina Diaz	2

  -
- Ventas por producto:

  SELECT p.nombre, SUM(cantidad) AS total_vendido
  FROM Ventas v
  JOIN Productos p ON v.id_producto = p.id_producto
  GROUP BY p.nombre;

nombre	total_vendido
Audifonos	4
Disco SSD	2
Impresora	3
Laptop	4
Monitor	2
Mouse	3
Silla Gamer	1
Tablet	1
Teclado	3
Webcam	4

  -

Consultas anidadas

Las subconsultas permiten resolver problemas mas complejos.

- Clientes con más de una compra

```
SELECT nombre
FROM Clientes
WHERE id_cliente IN (SELECT id_cliente
                     FROM Ventas
                     GROUP BY id_cliente
                     HAVING COUNT(*) > 1
                     );
```

nombre
Ana Torres
Juan Perez
Maria Lopez
Carlos Soto
Camila Rojas
Pedro Gonzalez
Daniela Muñoz
Luis Herrera
Valentina Diaz
Sebastian Morales

- Producto más vendido

```
SELECT nombre
FROM Productos
WHERE id_producto = (SELECT id_producto
                     FROM Ventas
                     GROUP BY id_producto
                     ORDER BY SUM(cantidad) DESC
                     LIMIT 1);
```

	nombre
1	Webcam

- Cliente que más gasto

```
SELECT nombre
FROM Clientes
WHERE id_cliente = (SELECT v.id_cliente
                    FROM Ventas v
                    JOIN Productos p ON v.id_producto = p.id_producto
                    GROUP BY v.id_cliente
                    ORDER BY SUM(p.precio * v.cantidad) DESC
                    LIMIT 1);
```

nombre
Camila Rojas

Creación y manipulación de tablas

Administramos las tablas con comandos DDL y DML.

- Agregar columna Stock a productos

```
-- Agregar columna stock
ALTER TABLE Productos
ADD COLUMN stock INTEGER;
```

- Actualizar stock de 1 producto y de todos los productos

```
--Actualizar stock a Laptop = id 1
UPDATE Productos
SET stock = 50
WHERE id_producto = 1;

-- Actualiza todos los stock
UPDATE Productos
SET stock = 20;
```

- Actualizar stock después de una venta, esto es una simula la reducción del inventario cuando se realiza una venta.

```
--Simulacion stock despues de una venta
UPDATE Productos
SET stock = stock - 1
WHERE id_producto = 1;
```

- Trigger Automático para descontar productos después de una venta

```
-- Trigger Automatico
CREATE TRIGGER actualizar_stock
AFTER INSERT ON Ventas
BEGIN
    UPDATE Productos
    SET stock = stock - NEW.cantidad
    WHERE id_producto = NEW.id_producto;
END;
```

- Eliminar producto

```
-- Eliminar producto obsoleto
DELETE FROM Productos
WHERE nombre = 'Webcam';
```

Eliminar un producto puede tener consecuencias importantes en la base de datos:

- Las ventas relacionadas pueden perder integridad
- Puede afectar reportes históricos
- En sistemas reales se recomienda usar soft delete.

Conclusión del proyecto

Este proyecto me permitió desarrollar un sistema básico de gestión de ventas utilizando SQL y base de datos relacionales.

A lo largo del desarrollo se implementaron:

- Creación de tablas
- Relaciones mediante claves primarias y foráneas
- Inserción de datos
- Consultas simples y complejas
- Funciones de agregación
- Subconsultas
- Manipulación de datos

El sistema permite registrar clientes, productos y ventas, y realiza análisis básico sobre la información almacenada

Este tipo de estructura es la base para sistemas más complejos como:

- ERP
- Sistema de inventario
- Plataformas de comercio electrónico

La estructura diseñada facilita el almacenamiento ordenado de la información y mejora significativamente la gestión de datos en comparación con el uso de planillas, reduciendo errores y permitiendo consultas más eficientes. Asimismo, el modelo propuesto sigue principios básicos de normalización, evitando redundancia de datos y asegurando una base de datos clara, coherente y escalable.

En conclusión, este proyecto demuestra cómo una base de datos relacional simple puede resolver necesidades reales de gestión de información en una empresa, permitiendo registrar operaciones, mantener control de los productos y generar información útil para la toma de decisiones. Además, constituye una base sólida para futuras mejoras, como la incorporación de reportes más avanzados, automatización del control de stock o integración con sistemas más complejos.